

AA Seminar 2

Vouchere de Vacanță

Problema este echivalentă cu problema rucsacului nefractionară

În cazul nostru, S = capacitatea rucsacului, iar costul destinațiilor pare cu cel al valorilor obiectelor și greutatea acestora

Din acest motiv, ne sortăm după raport

Algoritm

1. Sortăm locațiile desc. după cost
2. Parcurgem lista și adăugăm locațiile una câte una, cât timp suma nu depășește S

Justificare

S = bugetul

OPT = soluția optimă

ALG = suma algoritmului

Fie C_k prima locație respinsă

$$ALG + C_k > S$$

$$\Rightarrow ALG + C_k > OPT$$

$$ALG \geq C_1 \quad (\text{prima locație})$$

$$C_1 \geq C_k$$

$$\Rightarrow ALG \geq C_k$$

$$ALG + C_k \geq OPT$$

$$ALG + ALG \geq OPT$$

$$ALG \geq \frac{OPT}{2}$$

Camioane și transporturi

a) Contra exemplu

$G = 4$ (capacitatea unui camion)

$[3, 3, 1, 1]$

camion 1 : 3

camion 2 : $3 + 1 = 4$

camion 3 : 1

$$ALG = 3$$

$$OPT = 2$$

b) algorithm 2 approximativ

=> problema minimizare

$$OPT \geq \text{val} \left[\frac{w_1 + w_2 + \dots + w_n}{G} \right] \geq \frac{m}{G}$$

ALG = 2K comi vane

$$\underbrace{C_1, C_2, \dots, C_{2i-1}}_{> G}, \underbrace{C_{2i}, \dots, C_{2k-1}}_{> G}, \underbrace{C_{2k}}_{> G} \quad \text{red } k \cdot G$$

In orice două camioane, marcatura lor e mai mare decât 6 (nu a început în primul)

$$w_1 + w_2 + \dots + w_n > K \cdot G$$

$$\Rightarrow \text{OPT} > K \quad 1.2$$

$\Rightarrow 2OPT > 2K$
"
ALG

$$ALG = 2K + 1$$

$$\text{ALG: } \underbrace{C_1, C_2, \dots, C_{2k-1}}_{> G}, \underbrace{C_{2k}, \dots, C_{2k+1}}_{> G}, \underbrace{C_{2k+1}}_{> 1}$$

OPT: 1 cam
1 cam
1 cam
1 plin orhi
1 cel putin

OPT: $K+1$ comparisons

La doi pari, facuti de ALG, OPT face macor 1 ras

Moș, Crăciun

a) nu putem face sortare $\Rightarrow$ avem nevoie de $n \log m$
sau sortarea ar fi $n \log n$ ($n > m$)

1. Iterăm prin cadouri

2. Dam cadoul copilului cu suma cadourilor minimă

Pt $i = 1, m$

$C[i] = \text{null}$

$W[i] = 0$

Pt $j = 1, n$ // iterăm prin cadouri

Identific K cu $W[K]$ minim // $O(1)$ dacă țin minte

copiii într-o coadă de prioritate

$C[K].add(j)$

$W[K] += val(j)$ // $O(\log m)$ update PQ

Identific K - cu $W[K]$ minim

Alg va avea profitul de $W[K]$

$$b) \quad w[k] \geq w[0] - \text{val}(i)$$

↳ e mai mare deoarece $w[0]$ a primit cadoul

$$c) \quad \text{ALG} \geq \frac{\text{Val}}{2m}$$

$$\text{ALG} = w[k]$$

$$w[k] \geq w[1] - \text{val}(i_1)$$

$$w[k] \geq w[2] - \text{val}(i_2)$$

...

$$w[k] \geq w[k] - \text{val}(i_k)$$

...

$$w[k] \geq w[m] - \text{val}(i_m)$$

$$\hline (+) \quad \text{val}(i) < \frac{\text{Val}}{2m}$$

$$m \cdot \text{ALG} \geq \text{Val} - [\text{val}(i_1) + \text{val}(i_2) + \dots + \text{val}(i_m)]$$

$$> \text{Val} - \left(\frac{\text{Val}}{2m} + \frac{\text{Val}}{2m} + \dots + \frac{\text{Val}}{2m} \right)$$

$$\Leftrightarrow m \cdot \text{Alg} > \text{Val} - \frac{\text{Val}}{2}$$

$$\Leftrightarrow \text{Alg} > \frac{\text{Val}}{2m}$$

$$d) \quad \text{OPT} < \frac{\text{Val}}{m}$$

$$\frac{1}{2} \text{OPT} < \frac{\text{Val}}{2m} = \text{ALG}$$

Seminar 2

Stefan Popescu

April 2023

Vouchere de Vacanță

(10p) George a intrat în concediu, și ca atare dorește să consume cât mai mult din pachetul său de vouchere de vacanță. El și-a făcut o listă cu n locații unde poate cheltui din suma alocată. Știind că George nu are de gând să cheltuiască din banii proprii decât pe transportul între locații și că el se poate deplasa între oricare două locații, ajutați-l pe George să găsească o listă de locații pe care să le viziteze astfel încât să își folosească voucherele cât mai eficient.

Cerințe:

Se cere un algoritm 1/2-aproximativ care rezolvă problema lui George în timp eficient din punct de vedere al complexității timp. (pseudocod + justificare)

Algoritmul va avea ca output lista de locații și suma totală ce va fi cheltuită.

Notații și indicații:

- Notăm cu S suma de bani pe care o are George sub formă de vouchere.
- Locațiile sunt etichetate cu numerele $1, 2, \dots, n$. Costurile celor n locații sunt notate cu $C_1, C_2, \dots, C_n$
- Suma de bani S este alocată pe un card, nu sub formă de tichete. Din cont se poate extrage orice sumă disponibilă.
- Se lucrează cu numere întregi.

Camioane și Transporturi

(15p) Avem n colete de transportat, fiecare având greutatea de $w_1, w_2, \dots, w_n$. Pentru a le transporta, putem folosi un număr de camioane, fiecare având capacitatea de transport G . presupunem că $w \leq G$, pentru orice i . Ne dorim să minimizăm numărul de camioane folosite. Considerăm următorul plan de încărcare a camioanelor:

Odată ce avem la dispoziție un camion pentru a fi încărcat, iterăm prin mulțimea coletelor, încărcându-le în camion, până când dăm peste primul colet ce nu mai încapă. În acel moment considerat că am terminat de încărcat camionul curent și trecem la următorul camion, prima dată încărcând coletul care nu a mai încăput în cel precedent.

Cerințe:

- Arătați, printr-un exemplu simplu, că metoda de mai sus nu furnizează soluția optimă. (5p)
- Arătați totuși că soluția de mai sus este un algoritm 2-aproximativ pentru problema noastră. (10p)

Notații:

n - numărul de colete

G - capacitatea unui camion

W - Lista cu greutatea coletelor în ordinea în care sosesc spre a fi procesate.

OPT – soluția optimă care există: Numărul minim de camioane ce poate fi folosit pentru a transporta cele n colete de greutate totală G

ALG – soluția oferită de algoritmul nostru: numărul de camioane folosite conform algoritmului

$G[i]$ - greutatea cu care este încărcat camionul i în configurația dată de algoritmul nostru. Evident $G[i] \leq G$.

Cadourile lui Moș Crăciun

(30p) O clasă de elevi este formată din m copii. La această clasă Moș Crăciun aduce n cadouri, fiecare dintre ele având o valoare notată $val_1, val_2, \dots, val_n$. Moșul, grăbit fiind, lasă task-ul împărțirii cadourilor pe umerii profesorului de informatică. Acesta își alege următorul obiectiv pentru împărțirea cadourilor: să maximizeze valoarea cadourilor primite de către copilul care primește cel mai puțin. (Altfel spus, copilul cel mai “vitregit” în urma împărțirii să primească totuși cadouri de o valoare cât mai bună / Să existe un “spread” cât mai bun al cadourilor). În timp ce se gândește la o soluție de implementare, profesorul își amintește de la cursurile de algoritmică din facultate că astfel de probleme de optim sunt NP-hard! Fiind cuprins de groază (dar și de o nostalgie pentru cursurile de algoritmică din facultate) el vă cere vouă ajutorul.

Notății:

OPT – soluția optimă care există: Valoarea totală a cadourilor primite de copilul cel mai “vitregit” în o configurație optimă

ALG – soluția oferită de algoritmul vostru: Valoarea totală a cadourilor primite de copilul cel mai “vitregit” în urma algoritmului vostru

$C[k]$ - lista cadourilor primite de către copilul k la finalul algoritmului vostru

$W(K)$ – valoarea totală a cadourilor primite de către copilul k la finalul algoritmului vostru

$val(i)$ – valoarea cadoului i

$Val = \sum_{1 \leq i \leq n} val(i)$ – valoarea totală a cadourilor

Copiii vor fi indexați folosind variabile de forma k, q, k', q' , etc... Cadourile vor fi indexate folosind variabile de forma i, j, i', j' , etc...

Restricții: Considerăm că $n \geq m$ și că $val(i) \leq \frac{Val}{2m}$ pentru oricare cadou i .

Cerințe:

- Descrieți un algoritm $\frac{1}{2}$ aproximativ pentru problema cadourilor în complexitate $\mathcal{O}(n \log m)$ **(10p)**
- Fie k acel copil pentru care $ALG = W(K)$. Fie i ultimul cadou primit de un copil oarecare q ($q \neq k$). Care este relația între $W(K)$ și $W(Q) - val(i)$? Justificați. **(5p)**
- Pe baza punctului b) arătați că $ALG \geq \frac{Val}{2m}$ **(5p)**
- Demonstrați că algoritmul descris la punctul a) este $\frac{1}{2}$ aproximativ **(5p)**
- Dați un exemplu format din minimum 2 copii și 4 cadouri pentru care algoritmul vostru nu găsește soluția optimă. Spuneți care este soluția optimă. Spuneți care este soluția dată de algoritmul vostru. **(5p)**

Algoritmi Avansați

Seminar 2

Gabriel Majeri

1 Introducere

În seminarul trecut am discutat principalele tipuri de probleme computaționale și am văzut că, pentru unele probleme, putem calcula în mod eficient soluția optimă. În acest seminar vom investiga câteva probleme dificile din punct de vedere computațional, pe care nu le putem rezolva în mod eficient și pentru care putem cel mult să obținem rapid o soluție **aproximativă** [1].

2 Exerciții

1. (Vertex cover [2]) Problema acoperirii unui graf (neorientat) ne cere să găsim o **submulțime de noduri din graf** astfel încât orice muchie din graf să fie învecinată cu (cel puțin) un nod din această mulțime. Problema de optimizare asociată implică găsirea mulțimii de cardinal **minim**.

În cele ce urmează, vom presupune că avem un graf cu n și m muchii, reprezentat prin liste de adiacență.

1. Fiind dată o mulțime de noduri dintr-un graf, cum ați **verifica** că mulțimea dată este o acoperire? Ce complexitate de timp are soluția propusă?
2. Propuneți o soluție de tip brute-force care să rezolve problema (i.e. să verifice toate submulțimile posibile de noduri). Ce complexitate de timp are soluția?
3. Să presupunem că alegeți o muchie aleatoare din graf, cu capetele u și v . Având în vedere că într-o soluție (optimă) a problemei *vertex cover* trebuie „atinse” toate muchiile, ce puteți spune despre nodurile u și v în raport cu mulțimea de noduri care constituie soluția (optimă)?
4. Ne gândim la următorul algoritm pentru a construi o soluție la proble-

ma acoperirii:

- (a) Inițial, acoperirea noastră este mulțimea vidă $\emptyset$.
- (b) Cât timp mai sunt muchii neacoperite în graf:
 - i. Luăm (aleator) orice muchie din graf.
 - ii. Includem capetele muchiei în mulțimea care va reprezenta acoperirea noastră.
 - iii. Ștergem toate muchiile incidente la cele două noduri menționate.
- (c) Mulțimea de noduri construită este soluția la problema noastră.

Acest algoritm construiește o soluție *aproximativă* la problema de optim pe care vrem să o rezolvăm; în unele cazuri, s-ar putea să fie posibil să construim o acoperire folosind mai puține noduri.

Determinați factorul de aproximare al acestui algoritm, dați un exemplu când algoritmul obține soluția optimă și unul în care obține soluția cea mai proastă în raport cu soluția optimă.

2. (Maximum cut [3]) Fiind dat un graf neorientat (V, E) , problema tăieturii maxime ne cere să găsim o **submulțime de noduri din graf**, astfel încât numărul de muchii care o leagă pe aceasta de complementul ei să fie maxim. Cu alte cuvinte, vrem două mulțimi de noduri A și B , cu $A \cup B = V$ și $A = V \setminus B$.

Această problemă are aplicații în fizica statistică și în proiectarea de circuite electronice [4].

Un algoritm aproximativ care rezolvă problema, descris în [5], este următorul:

1. Alegem aleator două noduri diferite din graf, v_1 și v_2 .
2. Inițializăm $A := \{v_1\}$, $B := \{v_2\}$.
3. Cât timp mai sunt noduri care nu se află nici în A , nici în B :
 - (a) Luăm aleator un astfel de nod.
 - (b) Dacă are mai multe muchii care duc către A decât muchii care duc către B , îl punem în B ; altfel, în A .
4. Soluția finală sunt mulțimile A și B .

Determinați ce **complexitate de timp** are algoritmul propus, ce **factor de aproximare** oferă și dați un **exemplu** în care soluția aproximativă găsită de algoritm este cea mai îndepărtată de soluția optimă.

Referințe

- [1] Wikipedia contributors, *Approximation algorithm*, URL: https://en.wikipedia.org/wiki/Approximation_algorithm.
- [2] Wikipedia contributors, *Vertex cover*, URL: https://en.wikipedia.org/wiki/Vertex_cover.
- [3] Wikipedia contributors, *Maximum cut*, URL: https://en.wikipedia.org/wiki/Maximum_cut.
- [4] Francisco Barahona et al., „An Application of Combinatorial Optimization to Statistical Physics and Circuit Layout Design”, în *Operations Research* 36.3 (1988), pp. 493–513, URL: <http://www.jstor.org/stable/170992>.
- [5] Vijay V. Vazirani, *Approximation algorithms*, Springer, 2001, ISBN: 978-3642084690.